

Understanding Processor Resources

Understanding Processor Resources for Apache Spark

In this video, we learned how Apache Spark uses processor cores to run applications efficiently. Think of processor cores as workers in a factory. Each worker can handle a task, and the more workers you have, the more tasks you can complete at the same time. When you run an application in Spark, it assigns a certain number of these "workers" (or cores) to handle the tasks. If there are not enough workers available, some tasks will have to wait until others finish.

For example, imagine you have a small bakery with six bakers (cores) and two orders to fill. If the first order requires four bakers, they will start working right away. However, if the second order also needs four bakers, it will have to wait until at least two bakers from the first order finish their tasks. This is how Spark manages tasks and cores to maximize efficiency

CPU Cores as a Spark Resource

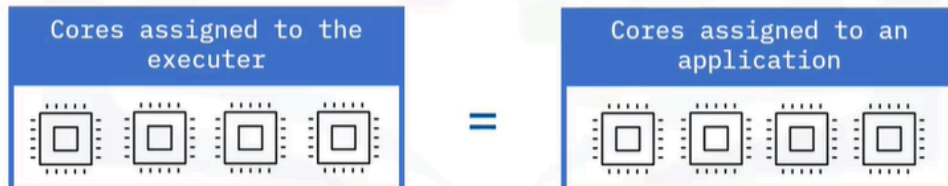
- Spark assigns CPU cores to driver and executor processes



- Just as with memory, CPU cores are a resource assigned to both the driver and executor processes.

CPU Cores as a Spark Resource

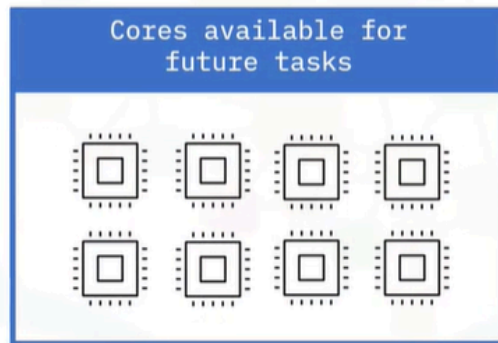
- Parallelism is limited by the number of cores available
- Executors process tasks in parallel up to the number of cores assigned to the application



- An executor can process tasks in parallel only up to the number of cores assigned to the application.

CPU Cores as a Spark Resource

- After processing, CPU cores become available for future tasks



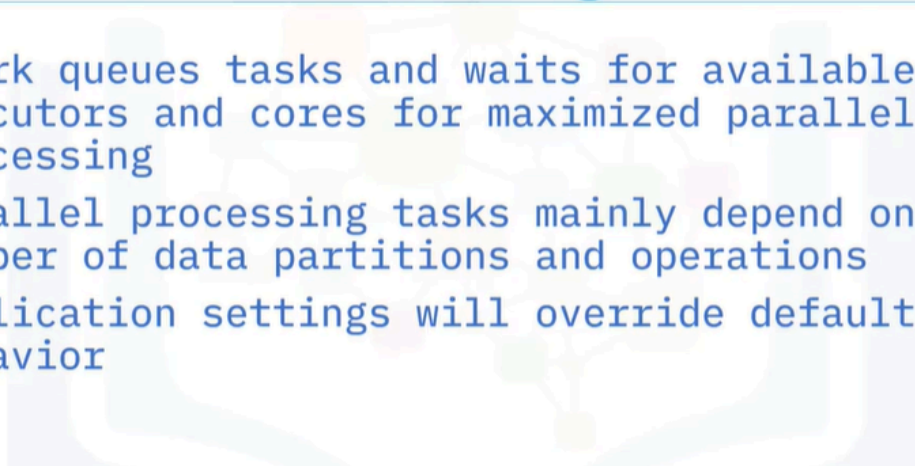
- Once the application tasks finish processing, the cores no longer in use return to the available pool.

CPU Cores as a Spark Resource

- Workers in the cluster contain a limited number of cores
- If no cores are available to an application, the application must wait for currently running tasks to finish

- The workers in the cluster contain a limited number of cores. If no cores are available to an application, the application must wait for currently running tasks to finish.

Default CPU Core Usage

- Spark queues tasks and waits for available executors and cores for maximized parallel processing
 - Parallel processing tasks mainly depend on the number of data partitions and operations
 - Application settings will override default behavior
- 
- When Spark launches an application and creates tasks, Spark places those tasks in a scheduling queue. Spark assigns tasks to any available executor. The default behavior is for each task that is ready, to be scheduled to an executor with available cores to maximize parallelism. Any remaining tasks will have to wait in the queue until more cores become available. If you provided specific settings for the number of cores, such as configuring how many cores the executor uses, the application overrides the default behavior.

Setting Executor Cores on Submit

- Specifies the number of executor cores for a Spark standalone cluster *per executor process*

```
$ ./bin/spark-submit \  
  --class org.apache.spark.examples.SparkPi \  
  --master spark://<spark-master-URL>:7077 \  
  --executor-cores 8 \  
  /path/to/examples.jar \  
  1000
```

Setting Executor Cores on Submit

- Specifies the executor cores for a Spark standalone cluster *for the application*

```
$ ./bin/spark-submit \  
  --class org.apache.spark.examples.SparkPi \  
  --master spark://<spark-master-URL>:7077 \  
  --total-executor-cores 50 \  
  /path/to/examples.jar \  
  1000
```

- To configure executor cores on application submission, use the argument `--executor-cores` followed by the specific number of cores required for the application tasks to run on an executor. As always, if the executor does not have the required number of cores available, tasks do not start until the specified number of cores are free or another executor becomes available.
- Alternatively, the argument `--total-executor-cores` followed by a number is the total amount of cores throughout the cluster to use for the application, not per

executor process. Tasks will be scheduled to executors whenever they have a free core, until the total number of cores reaches the specified value.

Setting Worker Node Resources

```
# Start standalone worker with MAX 10Gb memory, 8
cores
$ ./sbin/start-worker.sh \
  spark://<spark-master-URI> \
  --memory 10G --cores 8
```

- By default, Spark uses all available memory minus 1GB and all available cores

- When starting a worker manually in a Spark standalone cluster, you can specify the number of cores the application uses by using the argument `--cores` followed by the number of cores. Spark's default behavior is to use all available cores. A common practice is to start one worker per node, which under default behavior, allows the worker to create one executor process with exactly the same number of threads as cores available to run tasks. If another process is running on the same machine as the worker, such as the master, then a good practice is to apply these settings to reduce the number of cores available to the worker. Specifying the total number of CPU cores can reserve processing time available for the master process so essential operations are not blocked and time out.

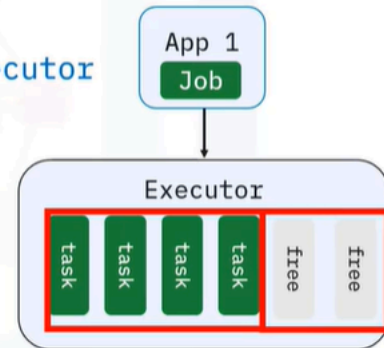
Core Utilization Example

A Spark standalone cluster with one worker node and six cores

- Submit App 1
- The application requests 4 cores per executor

```
$ ./bin/spark-submit \  
  --master spark://<spark-master-URL>:7077 \  
  --executor-cores 4 \  
  examples/src/main/python/pi.py \  
  1000
```

- App 1 occupies four cores
- Two cores remain free



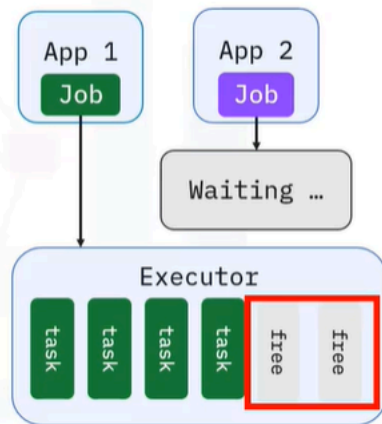
- Let's look at an example that uses a small standalone cluster with one worker node and six cores. This example uses two identical applications submitted to the same cluster in overlapping time, one right after the other. The first application is submitted to the cluster and requests four cores per executor. The executor is currently idle with six cores available. The first four tasks of the application are then scheduled to the executor and begin to run. Two cores of the executor remain available.

Core Utilization Example

- Submit App 2 using the same cluster
- The application requests 4 cores per executor

```
$ ./bin/spark-submit \  
--master spark://<spark-master-URL>:7077 \  
--executor-cores 4 \  
examples/src/main/python/pi.py \  
1000
```

- Two cores are available
- App 2 must wait until two additional cores are available



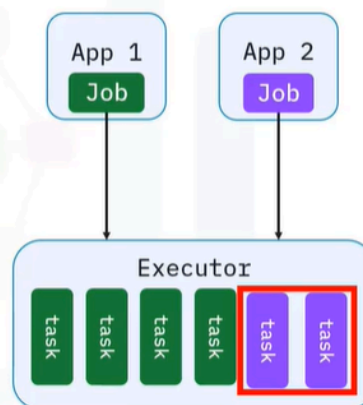
- While the first application is running, the second application is submitted. The second application also requests 4 cores from the executor. Because tasks that belong to the first application are still running and are using 4 cores, only 2 cores are available. Therefore, the second application must wait until the executor has at least four cores available. As soon as any two tasks from the first application complete and four cores become available, the second application can begin.

Core Utilization Example

- Start App 2 and request two cores

```
$ ./bin/spark-submit \  
--master spark://<spark-master-  
URL>:7077 \  
--executor-cores 2 \  
examples/src/main/python/pi.py \  
1000
```

- App 1 and App 2 now run simultaneously



- What if you start the second application and request only two cores to run two tasks? Using this scenario, you can submit a second application, and two tasks immediately start. However, the second application might take longer to finish because of running only two tasks at a time. Maximizing Spark application performance can be a tricky balance between configuration of application workloads and cluster resources and requires careful tuning of both the application and the clusters.

Summary

In this video, you learned that:

- Spark assigns CPU cores to driver and executor processes during application processing
- Executors process tasks in parallel according to the number of cores available or assigned by the application
- If using the Spark Standalone cluster manager, you can specify the total memory and CPU cores workers can use